МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп’ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ №2
Варіант 4

на тему «Функції вищого порядку, lambda, композиція»

Виконав:
студент групи ФеП-22
Корецький Андрій Вікторович

Перевірив:
аспірант
Остріков Данило Андрійович

Львів – 2026

Мета роботи. опанувати map / filter / reduce, lambda-вирази та функціональну композицію; побудувати конвеєр перетворень над списком записів без мутацій.

Теоретичні відомості

Функція вищого порядку приймає функції як аргументи або повертає функцію. Вбудовані map і filter застосовують функцію до кожного елемента або відбирають елементи за предикатом і працюють ліниво; functools.reduce згортає послідовність до одного значення.

lambda — анонімна функція з одного виразу; доречна для коротких ключів сортування, предикатів і перетворень.

Композиція compose(f, g)(x) = f(g(x)) дозволяє будувати конвеєр із маленьких чистих функцій; pipe застосовує функції зліва направо, juxt — кілька функцій до одного значення.

Індивідуальне завдання (варіант 4)

Тема «Замовлення ресторану». Запис: id, waiter, table, items. Конвеєр: обчислити суму чека, застосувати сервісний коефіцієнт для вибраної зони, відфільтрувати малі чеки, сформувати Top-N. reduce-агрегація: count, sum_checks, avg_check.

Хід роботи

1. Опис запису та композиція

Запис описано через TypedDict. Функції compose, pipe і juxt реалізовано через reduce.

```
class Record(TypedDict, total=False):
    """Замовлення: id, waiter, table, items (+ розрахункові поля)."""

    id: int
    waiter: str
    table: int
    items: list[OrderItem]
    zone: str
    check: float

def compose(*funcs: Callable[[Any], Any]) -> Callable[[Any], Any]:
    """compose(f, g, h)(x) == f(g(h(x))) — застосування справа наліво."""
    def composed(x: Any) -> Any:
        return reduce(lambda acc, f: f(acc), reversed(funcs), x)

    return composed
def pipe(value: Any, *funcs: Callable[[Any], Any]) -> Any:
    """pipe(x, f, g, h) == h(g(f(x))) — застосування зліва направо."""
    return reduce(lambda acc, f: f(acc), funcs, value)

def juxt(*funcs: Callable[[Any], Any]) -> Callable[[Any], tuple[Any, ...]]:
    """juxt(f, g)(x) == (f(x), g(x)) — кілька функцій над одним значенням."""
```

```
    return lambda x: tuple(f(x) for f in funcs)
```

Лістинг 1 — фрагмент файлу core.py

2. Чисті трансформації конвеєра

Кожен етап — фабрика, що повертає функцію Iterable[Record] → Iterable[Record]. Етапи створюють нові словники {**r, ...} і не змінюють вхідні записи. Зона залу визначається за номером столика (1–10 hall, 11–20 terrace, 21+ vip).

```
def with_check() -> Stage:
    """Додає поле ``check`` = сума чека."""
    return lambda recs: (
        {**r, "check": round(check_total(r.get("items", [])), 2)} for r in recs
    )
def apply_service(zone: str, factor: float) -> Stage:
    """Сервісний коефіцієнт для обраної зони (інші чеки без змін)."""

    def stage(recs: Iterable[Record]) -> Iterator[Record]:
        for r in recs:
            check = float(r.get("check", 0.0))
            new_check = check * factor if r.get("zone") == zone else check
            yield {**r, "check": round(new_check, 2)}

    return stage

def drop_small_checks(min_check: float) -> Stage:
    """Відфільтровує малі чеки (filter)."""
    return lambda recs: filter(lambda r: float(r.get("check", 0.0)) >= min_check, recs)

def sort_by_check_desc() -> Callable[[Iterable[Record]], list[Record]]:
    """Сортування за двома ключами: check спадно, id зростанням."""
    return lambda recs: sorted(
        recs, key=lambda r: (-float(r.get("check", 0.0)), int(r.get("id", 0)))
    )

def take(n: int) -> Callable[[Iterable[Record]], list[Record]]:
    """Top-N через itertools.islice."""
    return lambda recs: list(islice(recs, max(n, 0)))
```

Лістинг 2 — фрагмент файлу core.py

3. Складання конвеєра та reduce-агрегація

Конвеєр збирається через compose (читається знизу вгору). Сортування виконано за двома ключами: сума чека спадно, id зростанням. reduce_stats обчислює count, sum_checks, avg_check та max_check.

```
def reduce_stats(recs: Iterable[Record]) -> Stats:
    """count, sum_checks, avg_check, max_check через functools.reduce."""
    init: Mapping[str, float] = {"count": 0, "sum_checks": 0.0, "max_check": 0.0}

    def step(acc: Mapping[str, float], r: Record) -> Mapping[str, float]:
        check = float(r.get("check", 0.0))
        return {
            "count": acc["count"] + 1,
            "sum_checks": acc["sum_checks"] + check,
            "max_check": max(acc["max_check"], check),
        }
```

```
    acc = reduce(step, recs, init)
    count = int(acc["count"])
    total = round(acc["sum_checks"], 2)
    return {
        "count": count,
        "sum_checks": total,
        "avg_check": round(total / count, 2) if count else 0.0,
        "max_check": acc["max_check"],
    }

def build_pipeline(
    top_n: int = 3,
    zone: str = "terrace",
    factor: float = 1.1,
    min_check: float = 300.0,
) -> Callable[[Iterable[Record]], list[Record]]:
    """Конвеєр (читається знизу вгору, бо compose — справа наліво)."""
    return compose(
        take(top_n),
        sort_by_check_desc(),
        drop_small_checks(min_check),
        apply_service(zone, factor),
        with_check(),
        with_zone(),
        normalize_waiters(),
    )
```

Лістинг 3 — фрагмент файлу core.py

4. CLI-оболонка та тести

У app.py реалізовано читання JSON та аргументи командного рядка (argparse). 12 pytest-тестів перевіряють Top-N і тай-брейкер, відсутність мутацій, кожен етап окремо, агрегацію, compose / pipe / juxt та лінивість конвеєра.

```
def test_top_n(records: list[Record]) -> None:
    top = build_pipeline(top_n=3, zone="terrace", factor=1.1, min_check=300)(records)
    assert [r["id"] for r in top] == [3, 6, 2]
    assert [r["check"] for r in top] == [2200.0, 550.0, 440.0]

def test_tie_break_by_id(records: list[Record]) -> None:
    # чеки 2, 5, 8 рівні (440) -> порядок за id зростанням
    top = build_pipeline(top_n=6, zone="terrace", factor=1.1, min_check=300)(records)
    assert [r["id"] for r in top] == [3, 6, 2, 5, 8, 1]

def test_reduce_stats(records: list[Record]) -> None:
    stats = reduce_stats(prepare("terrace", 1.1, 300)(records))
    assert stats == {
        "count": 6,
        "sum_checks": 4440.0,
        "avg_check": 740.0,
        "max_check": 2200.0,
    }
```

Лістинг 4 — фрагмент файлу tests/test_core.py

Результати виконання

Результат запуску програми командою `python main.py --top 3 --zone terrace --factor 1.1`:

```
> python main.py --top 3 --zone terrace --factor 1.1
Top-3 чеків (зона 'terrace' x1.1, мін. чек 300.0):
  1. #3   Iryna        стіл 25  vip      чек = 2200.00
  2. #6   Andrii       стіл 18  terrace  чек = 550.00
  3. #2   Taras        стіл 12  terrace  чек = 440.00
Агрегація (reduce) по всіх допущених чеках:
  count      = 6
  sum_checks = 4440.00
  avg_check  = 740.00
  max_check  = 2200.00
```

Рисунок 1 — результат роботи програми (Lab_02)

Результат автоматизованого тестування `pytest -v` та статичних перевірок:

```
> black --check .
All done!
> ruff check .
All checks passed!
> mypy .
Success: no issues found
> pytest -v
============================= test session starts =============================
platform win32 -- Python 3.14.3, pytest-9.2.0, pluggy-1.7.0 -- C:\labs\Функціональне
    програмування\.venv\Scripts\python.exe
rootdir: C:\labs\Функціональне програмування\functional-programming\Lab_02
configfile: pyproject.toml
testpaths: tests
plugins: platformdirs-4.14.5
collecting ... collected 12 items
tests/test_core.py::test_check_total PASSED                              [  8%]
tests/test_core.py::test_table_zone_boundaries PASSED                    [ 16%]
tests/test_core.py::test_top_n PASSED                                    [ 25%]
tests/test_core.py::test_tie_break_by_id PASSED                          [ 33%]
tests/test_core.py::test_top_n_zero_and_larger_than_data PASSED          [ 41%]
tests/test_core.py::test_no_mutation PASSED                              [ 50%]
tests/test_core.py::test_stages_individually PASSED                      [ 58%]
tests/test_core.py::test_sort_and_take PASSED                            [ 66%]
tests/test_core.py::test_reduce_stats PASSED                             [ 75%]
tests/test_core.py::test_reduce_stats_empty PASSED                       [ 83%]
tests/test_core.py::test_compose_pipe_juxt PASSED                        [ 91%]
tests/test_core.py::test_pipeline_is_lazy_until_materialized PASSED      [100%]
============================== 12 passed in 0.05s ==============================
```

Рисунок 2 — результати тестів і перевірок якості коду (Lab_02)

Висновок

Побудовано функціональний конвеєр обробки замовлень ресторану з використанням map, filter, sorted, islice, lambda та композиції функцій. Усі етапи є чистими функціями й не змінюють вхідні дані, а конвеєр залишається лінивим до етапу сортування. Агрегацію виконано через functools.reduce. Код покрито тестами й проходить black, ruff і mypy.